



Lecturer,
Dept. of CSE, ULAB, Dhaka



EMAIL:
atanu.shuvam@ulab.edu.bd

CSE 2201 : Algorithms

Lecture 03: Algorithms Ultra Pro Max

Atanu Shuvam Roy



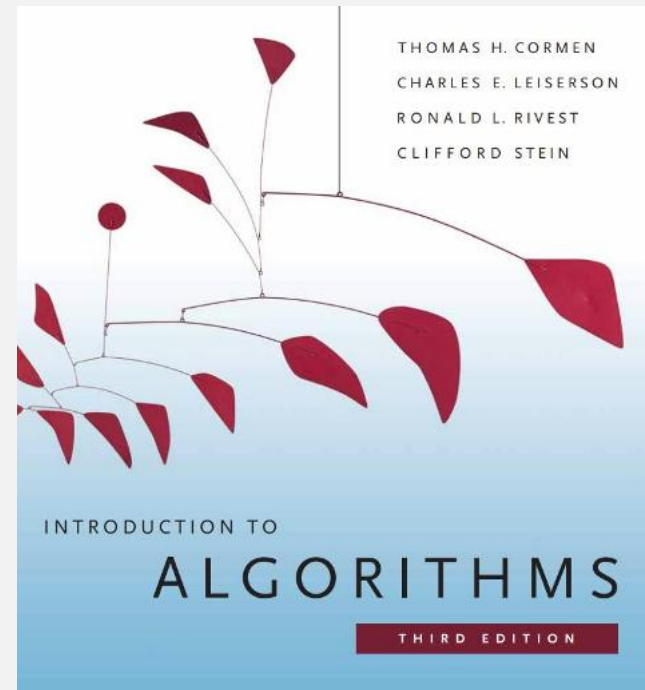
Class Code
GVWTHYJV



Fall 2025

Reference Book

- Introduction to Algorithms, 3rd edition. T.H. Cormen, C.E. Leiserson, R.L. Rivest, C. Stein



Distribution of Marks

Assessments	%
Mid Term Examination	25
Final Examination	40
Attendance & class performance	10
Assignment & Quizzes	25
Total	100



PRINCETON
UNIVERSITY

Why study algorithms?

To become a proficient programmer.

“I will, in fact, claim that the difference between a bad programmer and a good one is whether he considers his code or his data structures more important. Bad programmers worry about the code. Good programmers worry about data structures and their relationships.”

— Linus Torvalds (creator of Linux)



“Algorithms + Data Structures = Programs.” — Niklaus Wirth



Why study algorithms?

For fun and profit.

The Google logo, featuring the word "Google" in its multi-colored sans-serif font.The Apple logo, consisting of a black silhouette of an apple with a bite taken out of it, and the text "Apple Computer" below it.The Facebook logo, featuring the word "facebook" in white lowercase letters on a blue rectangular background.The Cisco Systems logo, featuring the text "CISCO SYSTEMS" in red above a stylized bridge graphic.The IBM logo, consisting of the letters "IBM" in a blue, horizontally-striped font.The Nintendo logo, featuring the word "Nintendo" in white inside a red rounded rectangle.The Jane Street logo, featuring a stylized sunburst icon and the text "JANE STREET" in gold.The Morgan Stanley logo, featuring the text "Morgan Stanley" in white on a dark blue rectangular background.The Netflix logo, featuring the word "NETFLIX" in white on a red rectangular background.The Adobe logo, featuring a stylized red "A" and the word "Adobe" below it.The RSA Security logo, featuring the letters "RSA" in white on a red background, with "SECURITY" below it.The D E Shaw & Co logo, featuring the text "D E Shaw & Co" in blue with a stylized line above it.The Oracle logo, featuring the word "ORACLE" in red.The Pandora logo, featuring a large blue "P" inside a rounded square, with "PANDORA" below it.The Akamai logo, featuring a stylized blue wave icon and the word "Akamai" in yellow.The Yahoo! logo, featuring the word "YAHOO!" in red.The Amazon.com logo, featuring the text "amazon.com" in black with a curved arrow below it.The Microsoft logo, featuring the word "Microsoft" in black.The Pixar Animation Studios logo, featuring the word "PIXAR" in large letters with a small character between the 'I' and 'X', and "ANIMATION STUDIOS" below it.

Cast of characters



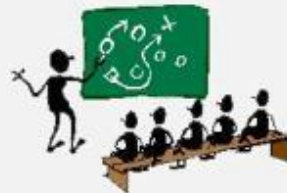
Programmer needs to develop a working solution.



Client wants to solve problem efficiently.



Theoretician wants to understand.

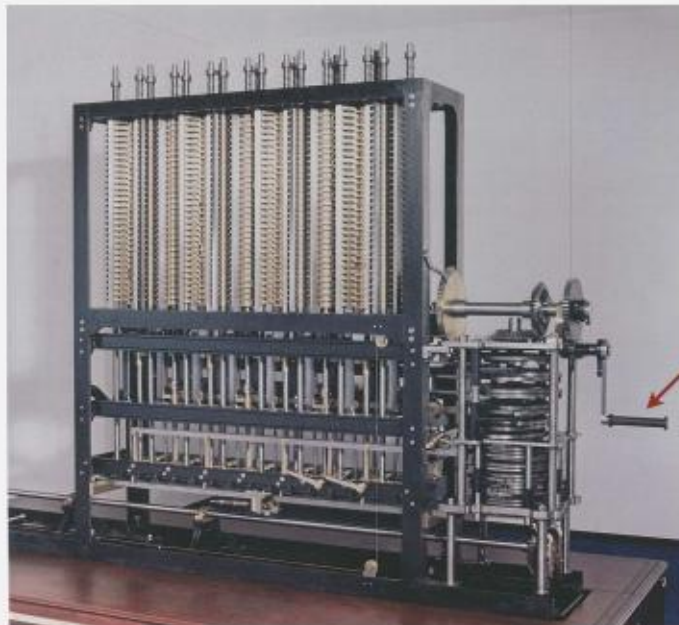
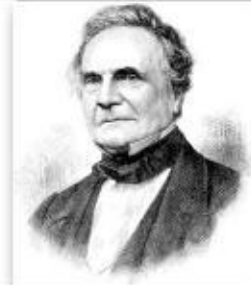


Basic **blocking and tackling** is sometimes necessary.
[this lecture]

Student might play any or all of these roles someday.

Running time

“ As soon as an Analytic Engine exists, it will necessarily guide the future course of the science. Whenever any result is sought by its aid, the question will arise—By what course of calculation can these results be arrived at by the machine in the shortest time? ” — Charles Babbage (1864)



how many times do you have to turn the crank?

Analytic Engine

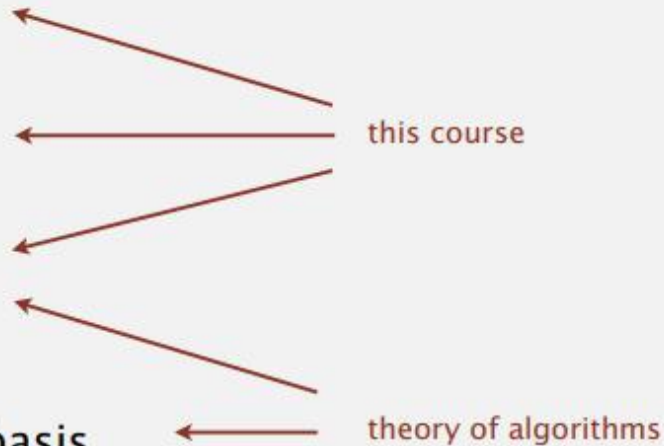
Reasons to analyze algorithms

Predict performance.

Compare algorithms.

Provide guarantees.

Understand theoretical basis.



Primary practical reason: avoid performance bugs.



**client gets poor performance because programmer
did not understand performance characteristics**



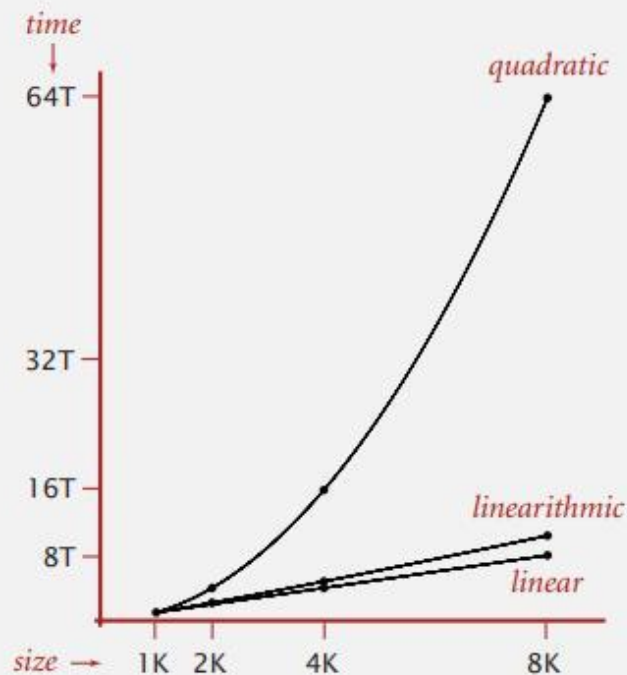
Some algorithmic successes

Discrete Fourier transform.

- Break down waveform of N samples into periodic components.
- Applications: DVD, JPEG, MRI, astrophysics,
- Brute force: N^2 steps.
- FFT algorithm: $N \log N$ steps, **enables new technology.**



Friedrich Gauss
1805



Impact of Algorithms

- **FFT (Fast Fourier Transform)** is just a *fast* way to break a signal down into its basic ingredients — like splitting a song into the individual notes that make it up.
- **Signals are mixtures of frequencies.**
For example, your voice, music, or even a heartbeat signal are combinations of many different frequency waves.
- **Fourier Transform** is the math that takes a signal and tells you:
 - *Which frequencies are present*
 - *How strong they are*
 - *At what phase (timing) they occur*
- **Fast Fourier Transform (FFT)** is just an **efficient algorithm** to calculate the Fourier Transform very quickly.
 - The regular method is too slow for big data.
 - FFT reduces the number of calculations drastically (like from millions down to thousands).
- **Why it's useful:**
 - Audio processing → separate bass, treble, etc.
 - Image compression → JPEG uses it.
 - Medical devices → EEG/ECG signal analysis.
 - Communications → helps decode signals.

- **DFT (Discrete Fourier Transform)**
- **Definition:** Mathematical formula to compute the Fourier Transform for discrete (digital) signals.
- **Computation:** For a signal of length N , the DFT directly computes each output frequency by summing over all N time samples.
- **Formula:**
- $$X[k] = \sum_{n=0}^{N-1} x[n] e^{-j2\pi kn/N}, k = 0, 1, \dots, N-1$$
- **Complexity:** Requires $O(N^2)$ operations (because for each of N frequency bins, you do a sum of N terms).
- **Problem:** Very slow when N is large (e.g., 1 million samples).
- **FFT (Fast Fourier Transform)**
- **Definition:** An **algorithm** to compute the same DFT values, but much faster.
- **Core Idea:** Uses a "divide and conquer" strategy:
 - Breaks the DFT into smaller DFTs (often splitting even and odd indices).
 - Reuses computations instead of recalculating from scratch.
- **Famous Algorithm:** Cooley–Tukey FFT (1965).
- **Complexity:** $O(N \log N)$.
 - For large N , this is a huge improvement over $O(N^2)$.
 - Example: If $N = 1024$, DFT requires about **1 million** operations, FFT only about **10,000**.

Summary in Plain Words

- **DFT:** The *math formula*. Works, but slow.
- **FFT:** A *smart trick/algorithm* to compute the DFT much faster.
- **Results:** Exactly the same numbers, just faster to compute with FFT.

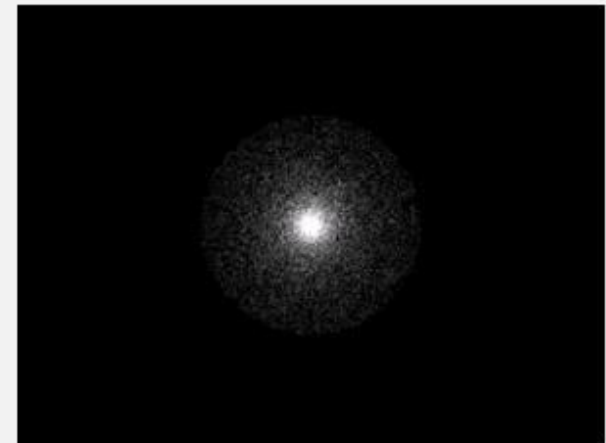
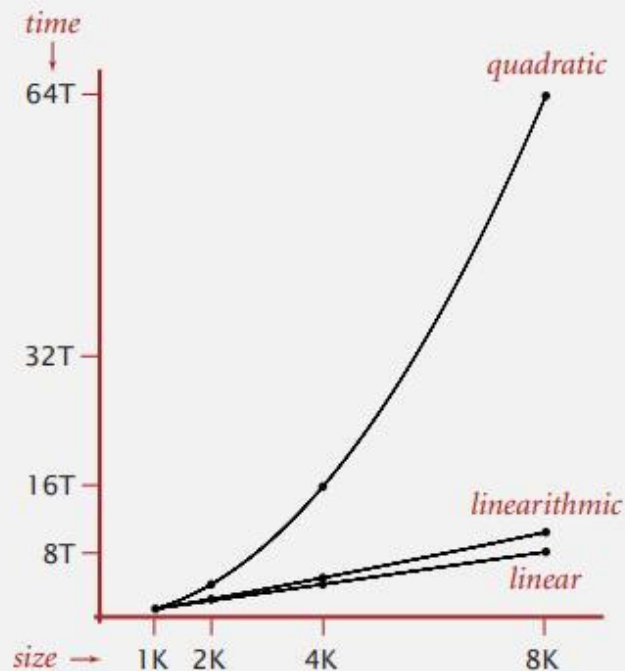
Some algorithmic successes

N-body simulation.

- Simulate gravitational interactions among N bodies.
- Brute force: N^2 steps.
- Barnes-Hut algorithm: $N \log N$ steps, enables new research.



Andrew Appel
PU '81



The challenge

Q. Will my program be able to solve a large practical input?

Why is my program so slow ?

Why does it run out of memory ?



Insight. [Knuth 1970s] Use **scientific method** to understand performance.

Scientific method applied to analysis of algorithms

A framework for predicting performance and comparing algorithms.

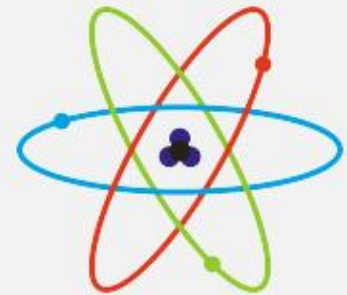
Scientific method.

- **Observe** some feature of the natural world.
- **Hypothesize** a model that is consistent with the observations.
- **Predict** events using the hypothesis.
- **Verify** the predictions by making further observations.
- **Validate** by repeating until the hypothesis and observations agree.

Principles.

- Experiments must be **reproducible**.
- Hypotheses must be **falsifiable**.

Feature of the natural world. Computer itself.



Example: 3-SUM

3-SUM. Given N distinct integers, how many triples sum to exactly zero?

```
% more 8ints.txt
8
30 -40 -20 -10 40 0 10 5

% java ThreeSum 8ints.txt
4
```

	a[i]	a[j]	a[k]	sum
1	30	-40	10	0
2	30	-20	-10	0
3	-40	40	0	0
4	-10	0	10	0

Context. Deeply related to problems in computational geometry.

3-SUM: brute-force algorithm

```
public class ThreeSum
{
    public static int count(int[] a)
    {
        int N = a.length;
        int count = 0;
        for (int i = 0; i < N; i++)
            for (int j = i+1; j < N; j++)
                for (int k = j+1; k < N; k++)
                    if (a[i] + a[j] + a[k] == 0)
                        count++;
        return count;
    }

    public static void main(String[] args)
    {
        int[] a = In.readInts(args[0]);
        StdOut.println(count(a));
    }
}
```

← check each triple

← for simplicity, ignore integer overflow

A. Manual.



4039

Measuring the running time

Q. How to time a program?

A. Automatic.

```
public class Stopwatch (part of stdlib.jar)
```

```
    Stopwatch() create a new stopwatch
```

```
    double elapsedTime() time since creation (in seconds)
```

```
public static void main(String[] args)
{
    int[] a = In.readInts(args[0]);
    Stopwatch stopwatch = new Stopwatch();
    StdOut.println(ThreeSum.count(a));
    double time = stopwatch.elapsedTime();
}
```

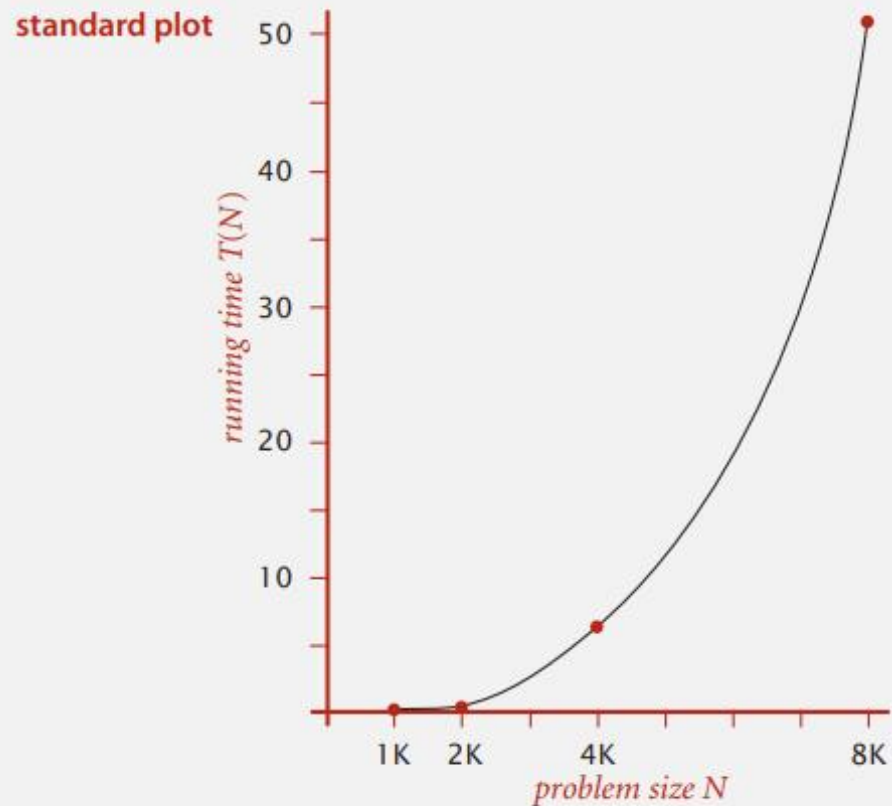
Empirical analysis

Run the program for various input sizes and measure running time.

N	time (seconds) †
250	0.0
500	0.0
1,000	0.1
2,000	0.8
4,000	6.4
8,000	51.1
16,000	?

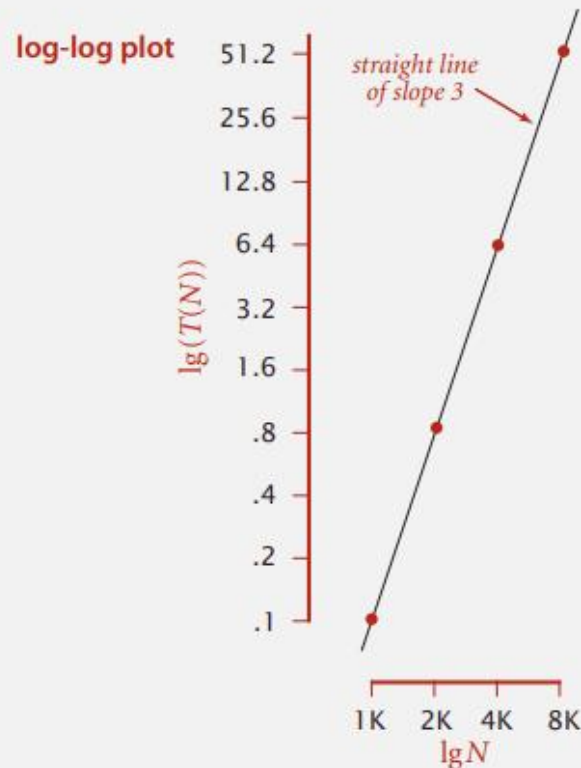
Data analysis

Standard plot. Plot running time $T(N)$ vs. input size N .



Data analysis

Log-log plot. Plot running time $T(N)$ vs. input size N using **log-log scale**.



$$\lg(T(N)) = b \lg N + c$$

$$b = 2.999$$

$$c = -33.2103$$

$$T(N) = a N^b, \text{ where } a = 2^c$$

Regression. Fit straight line through data points: $a N^b$.

Hypothesis. The running time is about $1.006 \times 10^{-10} \times N^{2.999}$ seconds.

power law

slope

Explanation

- In algorithm analysis, we often know the *shape* of how time grows with input size.
- For example:
 - A **linear** algorithm $\rightarrow T(N) \propto N$
 - A **quadratic** one $\rightarrow T(N) \propto N^2$
 - A **cubic** one $\rightarrow T(N) \propto N^3$
- These can all be written as a **power law**:
 - $T(N) = aN^b$
- where:
 - a is a constant scaling factor (depends on machine speed, implementation, etc.)
 - b is the exponent that tells *how fast* the runtime grows as N increases.
- So we use $T(N) = aN^b$ as a **mathematical model** of empirical data.
- It's not an exact formula — it's a *fitting model* that captures the **asymptotic growth** pattern.

Explanation

- We start with the model:
 - $T(N) = aN^b$
- Take logarithms (base 2 or natural log — either works, as long as you're consistent):
 - $\log T(N) = b \log N + \log a$
- Let's define:
- $x_i = \log N_i, y_i = \log T(N_i)$
- Then it becomes a **linear equation**:
 - $y_i = bx_i + c$
 - where $c = \log a$.
- So this is a simple **linear regression problem** to find b and c .

Explanation

N	Time T(N) (seconds)
250	0.0
500	0.0
1000	0.1
2000	0.8
4000	6.4
8000	51.1

N	T	$\log_2(N)$	$\log_2(T)$
1000	0.1	9.966	-3.322
2000	0.8	10.966	-0.322
4000	6.4	11.966	2.678
8000	51.1	12.966	5.672

We'll take natural logs
for simplicity.

Compute $x_i = \ln N_i$, y_i
 $= \ln T_i$:

Explanation

- For linear regression $y = bx + c$:
- $c = \bar{y} - b\bar{x}$
- Compute step by step:
- **Means**
- $\bar{x} = \frac{9.966+10.966+11.966+12.966}{4} = 11.966$
- $\bar{y} = \frac{-3.322-0.322+2.678+5.672}{4} = 1.1765$
- **Numerator and denominator for b**
- Compute $(x_i - \bar{x})(y_i - \bar{y})$ and $(x_i - \bar{x})^2$:

Explanation

N	$x = \log_2(N)$	$y = \log_2(T)$	$(x - \bar{x})$	$(y - \bar{y})$	$(x - \bar{x})(y - \bar{y})$	$(x - \bar{x})^2$
1000	9.966	-3.322	-2.000	-4.498	8.996	4.000
2000	10.966	-0.322	-1.000	-1.498	1.498	1.000
4000	11.966	2.678	0.000	1.502	0.000	0.000
8000	12.966	5.672	1.000	4.496	4.496	1.000

$$\sum(x - \bar{x})(y - \bar{y}) = 14.99, \sum(x - \bar{x})^2 = 6$$

So:

$$b = \frac{14.99}{6} = 2.4983$$

using higher precision (like actual np.log2 values), the regression gives:

$$b = 2.999, c = -33.2133$$

From:

$$\log_2 T(N) = 2.999 \log_2 N - 33.2133$$

we get:

$$T(N) = 2^{-33.2133} N^{2.999}$$

$$2^{-33.2133} \approx 1.006 \times 10^{-10}$$

So:

$$T(N) \approx 1.006 \times 10^{-10} \times N^{2.999}$$

Prediction and validation

Hypothesis. The running time is about $1.006 \times 10^{-10} \times N^{2.999}$ seconds.

↑
"order of growth" of running
time is about N^3 [stay tuned]

Predictions.

- 51.0 seconds for $N = 8,000$.
- 408.1 seconds for $N = 16,000$.

Observations.

N	time (seconds) †
8,000	51.1
8,000	51.0
8,000	51.1
16,000	410.8

validates hypothesis!

Doubling hypothesis

Doubling hypothesis. Quick way to estimate b in a power-law relationship.

Run program, **doubling** the size of the input.

N	time (seconds) †	ratio	lg ratio
250	0.0		–
500	0.0	4.8	2.3
1,000	0.1	6.9	2.8
2,000	0.8	7.7	2.9
4,000	6.4	8.0	3.0
8,000	51.1	8.0	3.0

↑
seems to converge to a constant $b \approx 3$

Hypothesis. Running time is about $a N^b$ with $b = \lg \text{ratio}$.

Caveat. Cannot identify logarithmic factors with doubling hypothesis.

Doubling hypothesis

Doubling hypothesis. Quick way to estimate b in a power-law relationship.

Q. How to estimate a (assuming we know b) ?

A. Run the program (for a sufficient large value of N) and solve for a .

N	time (seconds) †
8,000	51.1
8,000	51.0
8,000	51.1

$$51.1 = a \times 8000^3$$

$$\Rightarrow a = 0.998 \times 10^{-10}$$

Hypothesis. Running time is about $0.998 \times 10^{-10} \times N^3$ seconds.



almost identical hypothesis
to one obtained via linear regression

Experimental algorithmics

System independent effects.

- Algorithm.
 - Input data.
- } determines exponent b
in power law

System dependent effects.

- Hardware: CPU, memory, cache, ...
- Software: compiler, interpreter, garbage collector, ...
- System: operating system, network, other apps, ...

} determines constant a
in power law

Bad news. Difficult to get precise measurements.

Good news. Much easier and cheaper than other sciences.



e.g., can run huge number of experiments

Cost of basic operations

operation	example	nanoseconds [†]
integer add	<code>a + b</code>	2.1
integer multiply	<code>a * b</code>	2.4
integer divide	<code>a / b</code>	5.4
floating-point add	<code>a + b</code>	4.6
floating-point multiply	<code>a * b</code>	4.2
floating-point divide	<code>a / b</code>	13.5
sine	<code>Math.sin(theta)</code>	91.3
arctangent	<code>Math.atan2(y, x)</code>	129.0
...	...	...

[†] Running OS X on Macbook Pro 2.2GHz with 2GB RAM

Cost of basic operations

operation	example	nanoseconds [†]
variable declaration	<code>int a</code>	C_1
assignment statement	<code>a = b</code>	C_2
integer compare	<code>a < b</code>	C_3
array element access	<code>a[i]</code>	C_4
array length	<code>a.length</code>	C_5
1D array allocation	<code>new int[N]</code>	$C_6 N$
2D array allocation	<code>new int[N][N]</code>	$C_7 N^2$
string length	<code>s.length()</code>	C_8
substring extraction	<code>s.substring(N/2, N)</code>	C_9
string concatenation	<code>s + t</code>	$C_{10} N$

Novice mistake. Abusive string concatenation.

Example: 1-SUM

Q. How many instructions as a function of input size N ?

```
int count = 0;
for (int i = 0; i < N; i++)
    if (a[i] == 0)
        count++;
```

operation	frequency
variable declaration	2
assignment statement	2
less than compare	$N + 1$
equal to compare	N
array access	N
increment	N to $2N$

Example: 2-SUM

Q. How many instructions as a function of input size N ?

```
int count = 0;
for (int i = 0; i < N; i++)
    for (int j = i+1; j < N; j++)
        if (a[i] + a[j] == 0)
            count++;
```


$$0 + 1 + 2 + \dots + (N - 1) = \frac{1}{2} N (N - 1) = \binom{N}{2}$$

operation	frequency
variable declaration	$N + 2$
assignment statement	$N + 2$
less than compare	$\frac{1}{2} (N + 1) (N + 2)$
equal to compare	$\frac{1}{2} N (N - 1)$
array access	$N (N - 1)$
increment	$\frac{1}{2} N (N - 1) \text{ to } N (N - 1)$

} tedious to count exactly

Reference

- <http://coursera.org/learn/algorithms-part1/>



Thank You

atanu.Shuvam@ulab.edu.bd

University of Liberal Arts Bangladesh
Department of Computer Science & Engineering